Baho — Technical Report

Kinyarwanda-first, audio-first education on non-communicable diseases for rural Rwanda · Fiston Nshimiyandinze · BSc Software Engineering (FullStack), African Leadership University · Supervisor: Junior Turatsinze · November 2026
Live app: baho.filito.rw · Repository: github.com/fistonfle/baho

1. Overview

Baho is a progressive web app that teaches people with little health information in their own language how to prevent and live with hypertension, diabetes, heart disease and cancer. Learners pick topics, may fill a private health profile, and then follow a *study programme* of short lessons, each with an illustration, plain-Kinyarwanda text, audio and a quiz. Health workers write lessons, a specialist for each disease approves them, and an administrator publishes them. The app works offline after the first visit and runs on low-cost phones.

Architecture. React 18 + TypeScript (Vite) PWA with Redux Toolkit → REST API on Node 20 + Express (`/api/v1`) → PostgreSQL 16 (14 tables). In production the three parts run as a Docker stack behind Caddy (HTTPS); GitHub Actions tests, builds and deploys every push to `main`. The health profile is encrypted on the device and never sent to the server.

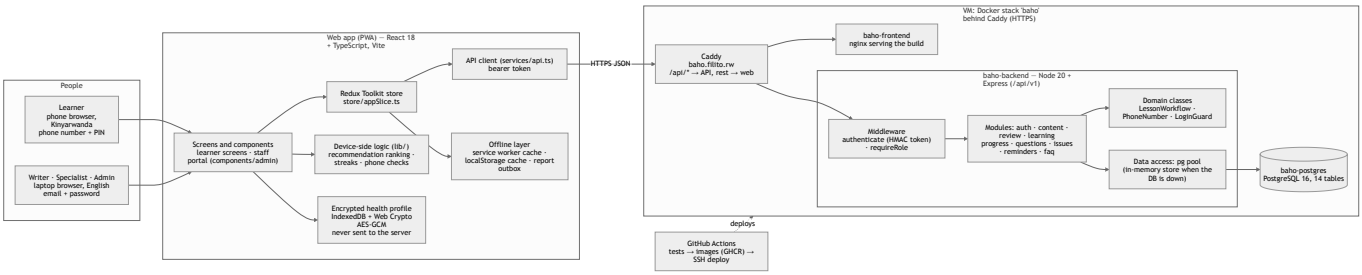


Figure 1. Architecture: web app, API and database in a Docker stack behind Caddy (source: `designs/architecture.mmd`)

2. Scope alignment and feature validation

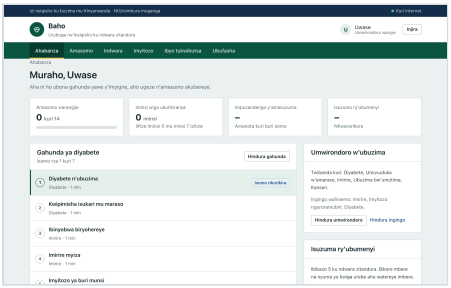
Requirements are those of the approved proposal (Tables 6 and 7). Evidence refers to the repository: screenshots in `designs/screenshots`, tests in `backend/tests` and `frontend/src/**/*.test.ts`, measurements in `docs/evidence`.

Planned feature	Intended purpose	Status	Evidence / justification of changes
FR01 Kinyarwanda content	Reach people who do not read English or French	**Completed**	All learner text in Kinyarwanda, reviewed through `docs/translations`; staff portal in English by request (easier to maintain and translate)
FR02 Audio playback	Learners with low literacy can listen	**Completed**	`AudioPlayer.tsx`: play, pause, replay, seek, speed; plays recorded MP3s, otherwise the device voice reads the lesson (screenshot 05)
FR03 NCD module	Awareness of hypertension, diabetes, heart disease	**Completed**	`NcdScreen.tsx`; cancer added; admins add new diseases (screenshots 08, 19)
FR04 Reminders	Habits: water, walking, medicine, check-ups	**Partial**	Reminders saved per account with browser notifications while the app is open; server Web Push not built (needs a push service and keys)
FR05 Illustrated exercises	Safe physical activity at home	**Completed**	`ExerciseScreen.tsx`: three levels, steps, illustration, audio, safety note (screenshot 10)
FR06 Offline-first	Weak rural connectivity	**Completed**	`sw.js` caches app and audio; lessons kept on device; tested offline with browser cache cleared (§4)
FR07 Onboarding: language, topics	Personalise from the first visit	**Partial**	Topics and name chosen; Kinyarwanda only, so no language choice yet (FR14)
FR08 Private risk self-assessment	Target content without collecting data	**Completed**	Conditions + six questions; AES-GCM in IndexedDB (screenshot 03)
FR09 Ranked content	Show the most relevant lessons first	**Completed**	`rankLessons` with the reason shown; unit-tested
FR10 Phone + PIN accounts, JWT	Most users have a phone, not an email	**Completed (changed)**	Phone + 4–6 digit PIN, lockout after 5 failures; one HMAC-signed 12-hour token instead of access + refresh pair (simpler, no extra dependency)
FR11 Sync with the server	Keep progress across devices	**Partial**	Progress, quiz scores, reminders, questions stored on the server for signed-in learners; anonymised

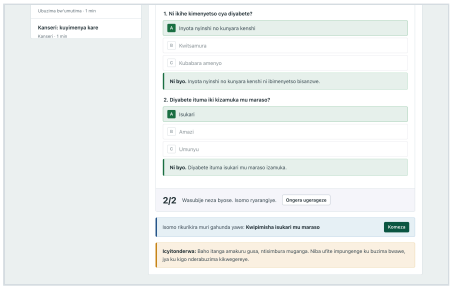
			usage analytics not built (privacy-first scope)
FR12 Admin content control	Publish, retire, edit; answer reports	**Completed**	Publish, unpublish, edit, delete; report status; FAQ management (screenshot 16)
FR13 Progress and streaks	Motivation to keep learning	**Completed**	Lessons done, streak, 7-day calendar, quiz average, knowledge-check gain
FR14 Future languages	Extend beyond Kinyarwanda	**Partial**	Every learner string is extractable and replaceable by script (`scripts/translations`); no runtime language switch
FR15 Responsive layout	Phones, tablets, laptops from one codebase	**Completed**	320–1920 px with no horizontal scroll; tab bar on phones (§4, screenshots 13–15)
FR16 Staff roles	Only trusted professionals write content	**Completed (changed)**	Admin creates writer, specialist (= reviewer) and admin accounts; specialists assigned to diseases; accounts created directly rather than by email invitation
FR17 Review before publishing	Medical accuracy	**Completed**	`LessonWorkflow`: specialist approval required (409 otherwise); no one reviews their own lesson (screenshot 20)
FR18 Offline FAQ with audio	Self-help without a connection	**Completed**	Searchable FAQ, audio answers, cached (screenshot 12)
FR19 Issue reports, queued offline	Feedback from the field	**Completed**	Outbox sends queued reports when online; admin updates status
NFR01, NFR03 Speed, size	≤ 3 s on mid-range Android; shell ≤ 5 MB	**Met**	1.35 s largest paint (mid-range), 2.71 s (low-end); 116 KB first visit
NFR04, NFR05 Security	HTTPS, hashed PINs, encrypted local data	**Mostly met**	HTTPS; scrypt (salted) instead of bcrypt; AES-GCM health profile; token in localStorage rather than an HttpOnly cookie
NFR06 Answers stay on device	Privacy	**Met**	No API accepts health-profile data
NFR08, NFR10 Compatibility	Current Chrome/Edge/Safari; 360–1920 px	**Met**	Chrome 156, Safari iOS 26.5; 6 widths without overflow
NFR09 Scalability	10,000+ users without re-engineering	**Met (local test)**	Stateless API, 5,400–21,000 req/s at 50 concurrent users, p99 ≤ 11 ms
NFR11 Access control, audit	Accountability for staff actions	**Partial**	Role checks on every staff route; reviewer and review time stored on each lesson; no separate audit-log table
NFR07 Usable without instruction	First-time smartphone users	**Not yet evaluated**	Needs the 30-participant study (SO3) planned after the prototype

Additions beyond the proposal (from supervisor feedback): a formal portal redesign replacing the first, generic-looking interface; phone sign-in prioritised because most users have no email; specialist approval per disease; English staff portal; a translation review pipeline.

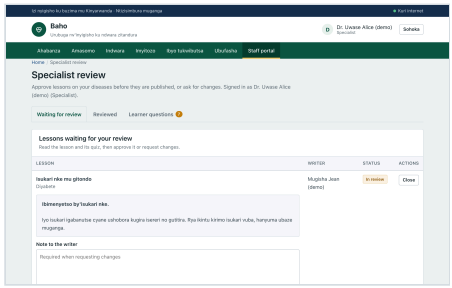
3. Technical functionality



Dashboard: programme, progress, ranked lessons



Lesson quiz graded by the server



Specialist reviewing a lesson

Frontend–backend interaction and data handling

- **Loading and caching.** On start the app requests categories, lessons, programmes and FAQs from the API, shows the copy kept in `localStorage` immediately, and replaces it when the response arrives; with no connection the cached or built-in content is used.

- **Accounts.** `POST /auth/login` takes a phone number or an email in one field. The server normalises the phone number, checks the lockout, verifies the scrypt hash and returns an HMAC-signed token with the user's role. Every protected route runs `authenticate` and `requireRole`.
- **Lessons and review.** A writer's lesson is saved as `review`. If its disease has a specialist, the lesson appears in that specialist's queue with the full text and quiz answers; approving sets `approved`, `reviewed_by` and `reviewed_at`. Only then can an admin publish it; earlier attempts return 409.
- **Quizzes and progress.** Answers are sent to `POST /quiz-attempts` and graded on the server, so a learner cannot alter their score; the best score per lesson is kept and the programme moves to the next lesson.
- **Privacy.** The health profile is encrypted with a non-extractable AES-GCM key stored in IndexedDB; only derived topic tags are used, on the device, for ranking.

4. Testing approaches and results

Six approaches were used (details: `docs/testing.md`). Each found something that was then improved.

Approach	Why and how	Results	Improvements made
Unit	Rules must hold in every case; Node test runner and Vitest	40 tests pass (workflow, phone/PIN, lockout, ranking, risk tags, streaks, store)	Streak split into a pure function; lockout made a class with an injectable clock
Integration (API)	Real Express app called over HTTP	17 tests pass: roles, phone sign-up, review/publish, quizzes, progress	Covers the whole specialist workflow end to end, so later changes cannot break it unnoticed
Different inputs	Valid, invalid and edge values (phone formats, PIN length, 5 wrong PINs, early publish)	Correct 400 / 403 / 409 / 429 responses with readable messages	Client-side checks mirror the server so mistakes show before sending
Performance	Lighthouse on the live site: mid-range phone, low-end phone (6× CPU, 3G), desktop	Performance 100 / 92 / 100; accessibility, best practices, SEO 100	Fixed layout shift (0.109 → 0), heading order, missing robots.txt
Compatibility and offline	Chrome 156 at 6 widths, Safari iOS 26.5 on two iPhones, offline with cache cleared	No overflow 320–1920 px; renders in Safari; first text in 2.9 s at 400 kbit/s	Found the app blank offline after a first visit; the service worker now caches the built files at install
Load	autocannon, 50 concurrent users, 15 s (local, not the shared server)	5,379–20,967 req/s; p99 ≤ 11 ms; 0 errors	Confirms the stateless API design for NFR09

Lighthouse (live)	Performance	Accessibility	Best practices	SEO	Largest paint	Layout shift	Blocking time
Mid-range phone	97 → 100	98 → 100	100	92 → 100	1.35 s	0.000	9 ms
Low-end phone	90 → 92	98 → 100	100	92 → 100	2.71 s	0.084 → 0.000	0 ms
Desktop	97 → 100	98 → 100	100	92 → 100	0.33 s	0.109 → 0.000	0 ms

5. Code quality, OOP and algorithms

Structure and principles

- **Separation of concerns:** backend modules follow routes → controller (HTTP only) → service (data); the frontend separates screens, reusable components (`ui.tsx`, `components/admin`), the store, the API client and pure logic (`lib/`).
- **Encapsulation:** `LessonWorkflow` owns all lesson-status rules, used by the content and review modules; `LoginGuard` keeps its failure map private (`#failures`).
- **Abstraction and polymorphism:** every service has a PostgreSQL and an in-memory implementation behind one contract (`withFallback`), so callers do not know which store answers and tests need no database.
- **Dependency injection:** `LoginGuard` takes its clock as a parameter, making time-based rules testable.
- **Single responsibility:** the 834-line staff screen was split into six components; two unused backend files

Algorithms and problem-solving

- **Lesson ranking** (`rankLessons`): score = 3·risk match + 2·topic match − 4·finished, sorted with newest first on ties; O(n log n). Transparent, so each card can say why it was suggested.
- **Risk tags:** each answer maps to topics, with "no" counting as the risk for healthy-habit questions; set union removes duplicates.
- **Streaks:** walk back day by day from today (or yesterday) over a set of active days; O(streak length) with O(1) lookups.
- **Review state machine:** `resolve(requested, current)` holds a publish request in review until approval.
- **Phone normalisation:** one regular expression accepts any written form and yields `+2507XXXXXXXX`.

were removed.
- **Quality gates:** strict TypeScript with unused-code errors, consistent naming (camelCase code, kebab-case CSS, snake_case SQL), comments that explain why, CI that runs all tests before any deploy.
- **PIN lockout:** counter per number with a time window, reset on success.
- **Offline caching:** network-first for pages, cache-first for built files and audio, install-time precaching read from `index.html`; an outbox replays reports when online.
- **Translation tooling:** TypeScript AST parsing extracts every learner string with its position, de-duplicates it, and writes corrections back safely.

6. Limitations and next steps

- Usability evaluation with 30 participants (SUS ≥ 70, task completion ≥ 80%) is the next phase (SO3).
- Server Web Push for reminders, refresh tokens in an HttpOnly cookie, an audit-log table and SMS verification of phone numbers.
- Recorded Kinyarwanda audio for each lesson (the device voice is used until then) and a clinical review of all lesson content.